Projet — Partie 1
Gestion intelligente du trafic urbain

Cours : IAD & Systèmes Multi-Agents
Cycle Ingénieur — Année 2025–2026

1. Contexte et objectifs

Les systèmes de gestion intelligente du trafic constituent un domaine d'application majeur de l'intelligence artificielle distribuée. Ce projet constitue un fil rouge pédagogique en deux parties : vous développez ici un agent unique capable d'optimiser les feux d'un carrefour isolé, en mobilisant les concepts vus en cours (modélisation de l'environnement, architectures d'agents, apprentissage par renforcement).

La Partie 2 du projet, abordée lors du module SMA, étendra ce travail à un réseau de carrefours interconnectés gérés par des agents coopératifs.

Objectif pédagogique : Concevoir, implémenter et analyser un agent intelligent apprenant par renforcement pour contrôler un feu de signalisation, en justifiant chaque choix de modélisation par rapport aux concepts du cours.

2. Description du problème

2.1 Scénario

On considère un carrefour à 4 branches (Nord, Sud, Est, Ouest). À chaque pas de temps discret, des véhicules arrivent aléatoirement sur chaque branche selon un processus de Poisson de paramètre λ_i (un paramètre par direction). L'agent contrôle le feu de signalisation : il décide quelle phase activer (N/S ou E/O autorisés à passer), avec la possibilité d'une phase de transition (orange).

2.2 Formalisation MDP — point de départ

Vous formaliserez entièrement le MDP. Les éléments suivants sont donnés à titre indicatif ; vous devrez les adapter, les justifier, et les étendre selon vos choix de conception :

Composant	Suggestion indicative — à justifier et adapter
Espace d'états S	Files d'attente discrétisées (ex. 0–4 véhicules / branche) + phase courante du feu. Taille : à discuter.
Espace d'actions A	{ Maintenir phase, Changer phase } — avec ou sans phase orange intermédiaire.
Fonction de récompense R	Typiquement basée sur le temps d'attente total ou le nombre de véhicules en attente. Votre formule doit être explicitée et justifiée.
Facteur d'actualisation γ	À choisir entre 0,9 et 1,0 selon l'horizon temporel visé.
Dynamique de transition	Stochastique (arrivées aléatoires). Matrice T non connue a priori → justifie le recours au Q-Learning.

3. Travail demandé

3.1 Modélisation et analyse (avant tout code)

1. Définissez formellement le MDP complet : états, actions, récompense, facteur γ. Justifiez chaque choix (discrétisation de l'espace, formule de récompense...).
2. Analysez les propriétés de l'environnement selon la classification du cours (observable, déterministe, épisodique, statique, discret, etc.).
3. Choisissez une architecture pour votre agent (réactive, délibérative, hybride, ou BDI) et justifiez ce choix au regard du problème. Expliquez pourquoi les autres architectures seraient moins adaptées.

3.2 Implémentation

4. Implémentez la simulation du carrefour (arrivées stochastiques, files d'attente, écoulement du trafic).
5. Implémentez l'algorithme Q-Learning avec les paramètres suivants à justifier : taux d'apprentissage α, facteur d'actualisation γ, politique d'exploration (ex. ε-greedy avec decay).
6. Implémentez une politique de référence (baseline) : feu à durée fixe, à alternance périodique, ou autre — au choix, à justifier.
7. Visualisez l'évolution de la table Q et/ou de la récompense cumulée au fil des épisodes.

3.3 Analyse et évaluation

8. Comparez la performance de votre agent Q-Learning avec la politique baseline sur au moins deux scénarios de trafic (ex. trafic équilibré vs charge asymétrique).
9. Analysez la convergence : courbes de récompense, stabilité de la politique, influence de α et ε.
10. Discutez les limites de votre modélisation et de l'approche tabulée (passage à l'échelle, explosion de l'espace d'états).

4. Contraintes techniques

4.1 Langage et outils

Langage imposé : Python 3.x. Bibliothèques autorisées :

- Simulation et calcul numérique : NumPy, Pandas
- Visualisation : Matplotlib, Seaborn
- Environnement de simulation (optionnel) : CityFlow, SUMO+TraCI, ou simulation custom
- RL (optionnel pour comparaison) : Stable-Baselines3, mais le Q-Learning tabulé doit être implémenté manuellement

Implémentation obligatoire : L'algorithme Q-Learning doit être implémenté from scratch (boucle d'apprentissage, mise à jour de la table Q, politique ε-greedy). L'utilisation d'une bibliothèque RL pour cette partie sera penalitée.

4.2 Organisation du code

Le dépôt Git doit contenir :

- README.md avec instructions d'installation et d'exécution
- Un module de simulation du carrefour (séparable de l'agent)
- Un module agent Q-Learning
- Un script d'entraînement et un script d'évaluation distincts
- Les courbes de convergence exportées (PNG ou PDF)

5. Livrables

Livrable	Description	Format / Délai
Rapport écrit	Modélisation MDP, architecture agent, analyse des résultats, discussion. Maximum 15 pages hors annexes.	PDF — J-7 avant soutenance
Code source	Dépôt Git ou archive ZIP. Code commenté, README inclus, scripts exécutables.	Git/ZIP — J-7 avant soutenance
Soutenance	Présentation de 15 min + démonstration live de l'agent + 10 min de questions.	Présentiel — date TBD

6. Conseils méthodologiques

Approche recommandée

Commencez par une version très simplifiée (2 files, 3 états par file, 2 actions) pour valider votre implémentation et observer la convergence. Étendez ensuite progressivement la complexité. Un agent qui fonctionne sur un problème simple et bien analysé vaut mieux qu'un agent ambitieux mais dont le comportement n'est pas compris.

- Versionez votre code dès le début (Git). Les commits réguliers montrent la progression du travail.
- Documentez chaque hypothèse de modélisation : pourquoi avez-vous discrétisé les files à 5 niveaux plutôt que 10 ? Pourquoi $\gamma = 0{,}95$?
- Les figures doivent être légendées et commentées dans le rapport. Une courbe sans analyse ne compte pas.
- La démonstration live est évaluée : prévoyez un mode de visualisation temps réel de votre agent.